Backend Documentation

Project:

Has 3 main features:

- Vehicle rental
- Ride booking
- carpool

System architecture:

ADBMS-Project/

Database/

- Schema.sql
- Seed.sql
- performance.sql

backend/

config/

- database.js

controllers/

//all controllers

middleware/

//middlwares

routes/

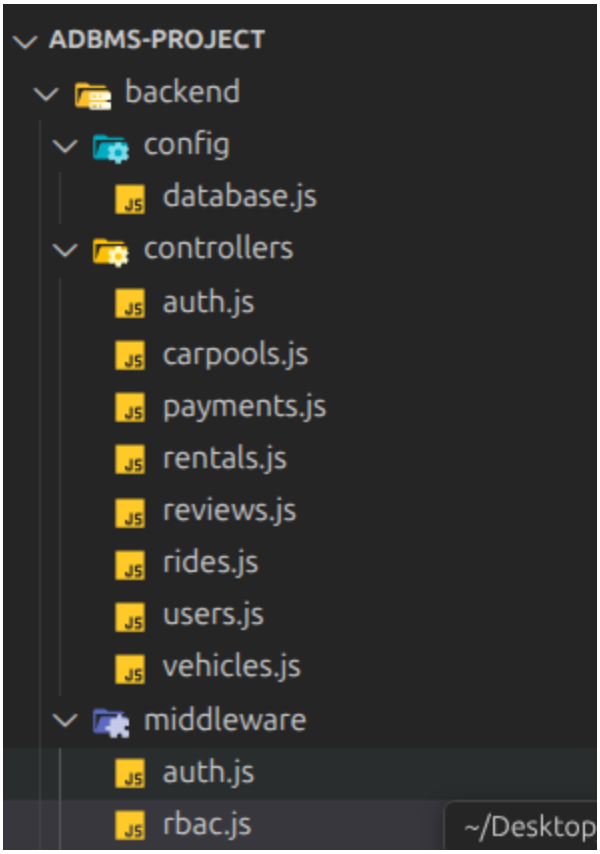
//all routes

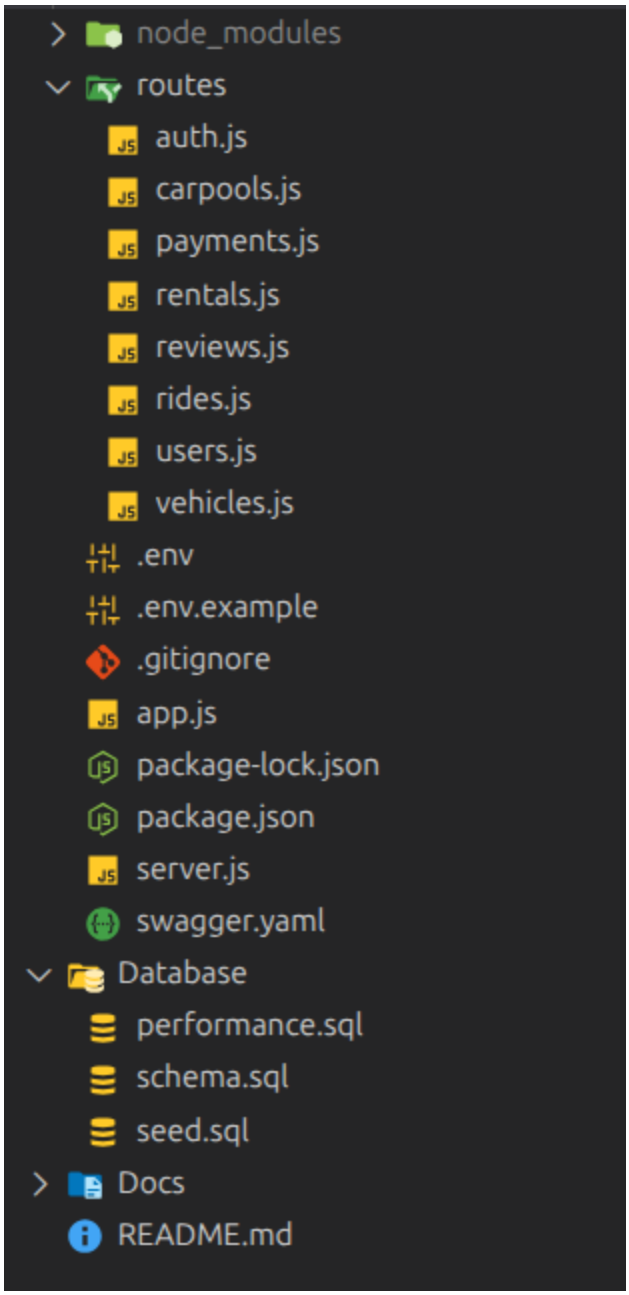
server.js

app.js

swagger.yaml

Better view:


ADBMS-PROJECT
backend
config
database.js
controllers
auth.js
carpools.js
payments.js
rentals.js
reviews.js
rides.js
users.js
vehicles.js
middleware
auth.js
rbac.js
~/Desktop


node_modules
routes
auth.js
carpools.js
payments.js
rentals.js
reviews.js
rides.js
users.js
vehicles.js
.env
.env.example
.gitignore
app.js
package-lock.json
package.json
server.js
swagger.yaml
Database
performance.sql
schema.sql
seed.sql
Docs
README.md

Request Lifecycle:

Step 1 : server.js
The entry point. It starts the server and checks the database connection before accepting any requests.

Step 2 : Global Middleware (app.js)
Then the request passes through 4 things that run on every request:

- helmet: adds security headers to the response
- cors: allows the frontend to talk to the backend
- express.json: reads the request body and turns it into a JavaScript object
- rateLimit: counts how many requests my IP has made. If limit exceeded it blocks

Step 3: Routing
Express looks at the URL and decides which route file to hand the request to.

- /api/v1/auth -> auth router
- /api/v1/rentals -> rentals router
- /api/v1/rides -> rides router

And so on

```
//eg: any request with /api/v1/routes is routed to authRoutes to handle
app.use('/api/v1/auth', authRoutes);
app.use('/api/v1/users', userRoutes);
app.use('/api/v1/vehicles', vehicleRoutes);
app.use('/api/v1/rentals', rentalRoutes);
app.use('/api/v1/rides', rideRoutes);
app.use('/api/v1/carpools', carpoolRoutes);
app.use('/api/v1/payments', paymentRoutes);
app.use('/api/v1/reviews', reviewRoutes);
```

Step 4 : Middleware on the route (authenticate + authorize)
Some routers have the authenticate/authorize middleware es that are defined in /backend/middleware/ .They run before any controller in that router.

Eg: routes/auth.js

```
//below routes rewquire authentication aswell
//in simpler words user must be logged in
router.get('/profile', authenticate, ctrl.getProfile);
router.put('/profile', authenticate, ctrl.updateProfile);
router.put('/change-password', authenticate, ctrl.changePassword);

module.exports = router;
```

Authenticate + authorize combo is used to diff between public vs protected routes.

Authenticate:

1. Reads the Authorization: Bearer <token> header
2. Verifies the JWT : if expired or fake, returns 401 immediately (access denied)
3. Fetches the user + their roles from the database and attaches it to req.user

Authorize:

```
//after authorization, check whether you are allowed to do etc...

const authorize = (...allowedRoles) => {
    return (req, res, next) => {
        if (!req.user || !req.user.roles) {
            return res.status(401).json({ error: 'Authentication required.' });
        }

        const hasRole = req.user.roles.some((role) => allowedRoles.includes(role));
        if (!hasRole) {
            return res.status(403).json({
                error: `Access denied. Required role(s): ${allowedRoles.join(', ')}`,
            });
        }

        next();
    };
};

module.exports = { authorize };
```

Defined in middleware/rbac.js

1. Looks at req.user.roles
2. If the user has the required role : continue
3. If not : return 403 Access Denied

The controller never runs if either of these fail.

Step 5: Controller

This is where the actual work happens. The database logic is also defined here.

1. Reads req.body and extracts key information
2. Runs the database query or transaction
3. Sends back a JSON response

Step 6: Database

The controller communicates to PostgreSQL through a connection pool (max 20 connections, defined in config/database.js).

- Simple operations use query() : grabs any free connection
- Transactions use getClient() : grabs a dedicated connection that stays locked until commit or rollback

```
//text = sql string and params = vals array []
//basically a wrapper so controller write query() instead of pool.query()..
const query = (text, params) => pool.query(text, params);

//get a dedicated connection(helps in transactions part) otherwise query sends whioever is free thar breaks trans
const getClient = () => pool.connect();
```

Step 7 : Response

- If app.js global error handler NOT triggered then: (no error)
- JSON sent back to client

```
//special 4 params
//explanation:
//When any controller calls next(err),Express skips all normal m
app.use((err, _req, res, _next) => {
    console.error('Unhandled error:', err);
    res.status(err.status || 500).json({
        error: err.message || 'Internal server error',
    });
});
```

Authentication Flow & RBAC Enforcement

middleware/auth.js :Verifies identity
middleware/rbac.js :Verifies permission

How are they used:

Part 1 :Registration

Endpoint: POST /api/v1/auth/register

The user sends email, password, phone, full_name, and optionally a role.

```
const { email, password, phone, full_name, role } = req.body;
```

1. Validates that required fields are present
2. Role is checked : only customer or driver are accepted. Anything else defaults to customer. Nobody can self-register as admin.

3. Password is hashed using bcrypt
4. A transaction begins:
   - INSERT INTO users : creates the user row
   - INSERT INTO user_roles : links the user to their role in a separate table

   Both succeed or both rollback together

5. A JWT token is generated and returned : the user is logged in right after registering
From code:

```
res.status(201).json({
    message: 'Registration successful',
    token,
    user: {
        user_id: newUser.user_id,
        email: newUser.email,
        full_name: newUser.full_name,
        role: selectedRole,
        created_at: newUser.created_at,
    },
});
```

6. If the email or phone already exists, PostgreSQL throws error code 23505 (unique violation) : caught and returned as a 409 Conflict

Part 2 : Login

Endpoint: POST /api/v1/auth/login

1. Fetches the user from the database by email, including their roles via a JOIN across three tables (users, user_roles, roles)
2. Checks is_active : if the admin has deactivated this account, returns 403 immediately
3. checks the submitted password against the stored hash
4. If everything passes, a JWT is signed containing only userId and email

```
const token = jwt.sign(
    { userId: user.user_id, email: user.email },
    //same secret when checked in middlewere/auth.js
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '7d' }
);
```

authorize Middleware (RBAC)

File: middleware/rbac.js

Runs after authenticate on routes that require a specific role.

authorize('driver') : only drivers allowed
authorize('admin') : only admins allowed
authorize('customer') : only customers allowed etc

```
//after authorization, check whether you are allowed to do etc...

const authorize = (...allowedRoles) => {
    return (req, res, next) => {
        if (!req.user || !req.user.roles) {
            return res.status(401).json({ error: 'Authentication required.' });
        }

        const hasRole = req.user.roles.some((role) => allowedRoles.includes(role));
        if (!hasRole) {
            return res.status(403).json({
                error: `Access denied. Required role(s): ${allowedRoles.join(', ')}`,
            });
        }

        next();
    };
};

module.exports = { authorize };
```

1. Looks at req.user.roles
2. If the user has the required role : continue
3. If not : return 403 Access Denied

Transaction Flows & Rollback Conditions

POST /auth/register
POST /rentals
POST /rides
POST /carpools/bookings

All transactions follow the exact same pattern:

getClient(): grab a dedicated connection from the pool
BEGIN: start the transaction
Run queries: all on the same connection
COMMIT: finalize everything at once
ROLLBACK: undo everything if anything fails
client.release() : always returns the connection to the pool

Transaction 1 : Register (POST /api/v1/auth/register)

Creates a new user and assigns them a role. Both must succeed together. A user row without a role entry would break all authentication.

```
const register = async (req, res, next) => {
    const client = await getClient();
    try {
        const { email, password, phone, full_name, role } = req.body;

        if (!email || !password || !phone || !full_name) {
            return res.status(400).json({ error: 'email, password, pho
        }
```

steps:

1. INSERT INTO users
2. INSERT INTO user_roles
3. COMMIT

Rollback conditions:

- Email already registered
- Phone no. already registered
- Any unexpected DB error -> goes to global error handler defined in app.js

Transaction 2 : Create Rental (POST /api/v1/rentals)

Books a vehicle for a date range and creates the payment record aswell.

Steps:

1. Lock row and check vehicl availablity in vehicles table
2. Check if vehicle is available or not
3. Check if vehicle already has a pending/active booking inthose dates
4. Calculate total amount
5. Insert row in rental_bookings
6. Insert row in rental_payments
7. commit

Rollback conditions

- Vehicle not found
- Vehicle is not available
- Date overlap with existing booking
- Start_date is in past (DB trigger checks this)

```
CREATE OR REPLACE FUNCTION fn_check_rental_start_date()
RETURNS TRIGGER AS $$
BEGIN
    --idhr current date se kam ni hoskta
    --completed/cancelled bookings ko allow karo (historical records)
    IF NEW.start_date < CURRENT_DATE AND NEW.status NOT IN ('completed', 'cancelled') THEN
        RAISE EXCEPTION 'cannot create booking with past start date.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

- End_date < start_date (DB check constraint on table)

FOR UPDATE use:
When two customers try to book the same vehicle at the same time, both transactions run SELECT FOR UPDATE on the vehicle row. PostgreSQL only allows one of them to hold the lock. The second request waits. Once the first commits, the second reads the vehicle , if it is now unavailable, it rolls back. This prevents double bookings.

Transaction 3 : Book Ride (POST /api/v1/rides)

Books a ride with a specific driver and vehicle, and creates the payment record.

Steps:

1. Verify that the driver_id provided has a driver role
2. Lock the row
3. Check if vehicle is available
4. Insert in ride_bookings
5. Insert in ride_payments
6. commit

Rollback conditions

- driver_id is not a registered driver
- Vehicle not found
- Vehicle is_available = false
- Any unexpected DB error (global error handler)

Trigger involvement

trg_ride_vehicle_availability fires after INSERT on ride_bookings. Because the ride is inserted with status = active, the trigger immediately sets vehicles.is_available = FALSE. This happens inside the same transaction.

Transaction 4 : Book Carpool (POST /api/v1/carpools/bookings)

Books one or more seats in a carpool offer and creates the payment record.

Steps:

1. Lock the row for update
2. Check status = ‘open’
3. Insert in carpool_bookings
4. Insert in carpool_payments
5. commit

Rollback conditions

- Carpool offer not found
- Status is not open
- Not enough seats (DB trigger)
- Passenger already booked this carpool (constraint defined in table)
- Any db error etc

Two triggers fire inside this transaction

trg_check_carpool_seats**:** fires before INSERT on carpool_bookings. It checks if available_seats >= seats_booked. If not, raises an exception which aborts the INSERT transaction rollbacks

trg_carpool_seats_update: fires after INSERT on carpool_bookings. It decrements available_seats on the offer. If available_seats drops to 0, it also changes the offer status to ‘full’.

Raw SQL & connection pooling strategy

Eaw sql queries are used throughout the code.
There are simple queries and dedicated client(transactional queries).

Examples:
1.

```
'SELECT * FROM carpool_offers WHERE carpool_id = $1 FOR UPDATE'
```

2.

```
const client = await getClient();
   try {
```

```
        const { carpool_id, seats_booked, payment_method } = req.body;
        if (!carpool_id || !seats_booked) return res.status(400).json({
error: 'carpool_id and seats_booked are required.' });

        await client.query('BEGIN');

        const offerResult = await client.query('SELECT * FROM
carpool_offers WHERE carpool_id = $1 FOR UPDATE', [carpool_id]);
```

Connection pooling strategy:

```
const pg = require('pg');

//connection pool
const Pool = pg.Pool;

const pool = new Pool({
 host: process.env.DB_HOST || 'localhost',
 port: parseInt(process.env.DB_PORT, 10) || 5432,
 database: process.env.DB_NAME || 'rideshare_db',
 user: process.env.DB_USER || 'postgres',
 password: process.env.DB_PASSWORD || 'postgres',
 max: 20,  //simultaneous connections
 idleTimeoutMillis: 30000,
 connectionTimeoutMillis: 5000,
});
```

Pg.pool instance in created when the app starts. Allows 20 simultaneous connections. A connection idle for 30 secs is closed and removed from pool. If no connection is free after 5 secs, the request errors out

Some helpers:

```
const query = (text, params) => pool.query(text, params);

//get a dedicated connection(helps in transactions part) otherwise query
sends whioever is free thar breaks transactions logic
const getClient = () => pool.connect();
```

Query picks any free connection from the pool and runs the query

getCLient grabs a dedicated connection from the pool used for transactions

Key Design Decisions & Tradeoffs

1. Raw SQL over ORM
Full control over queries, parameterised by default (`$1, $2`)

3. Roles in a junction table, not a column
Users can hold multiple roles simultaneously (customer + driver). Adding new roles needs no schema change.
Tradeoff: every role check needs a 3-table JOIN.

4. FOR UPDATE lock before booking
Prevents two concurrent requests from double-booking the same vehicle.
Tradeoff: second request must wait for the lock to be released

5. Vehicle availability managed by DB trigger
trg_ride_vehicle_availability auto-updates is_available on ride status changes : controller never touches the vehicles table directly.
Tradeoff: behaviour is invisible from controller code alone.

6. Carpool seat logic entirely in triggers
Seat validation and decrement happen at the DB level, guaranteeing correctness even under concurrency.